

Es hora de que pongas en práctica todo lo aprendido. 🧐

Te proponemos un ejercicio que te permitirá practicar la conexión a bases de datos utilizando JDBC, aplicar consultas SQL para recuperar y modificar datos, implementar el patrón Singleton y aplicar el patrón DAO para desacoplar la capa de acceso a datos.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😊

¡Manos a la obra!

1. Desafío 🎯

Desarrolla una **aplicación Java** que se **conecte a una base de datos MySQL** e implemente las siguientes funcionalidades utilizando **JDBC**:

- Crear **una tabla** llamada *PRODUCTOS* con los campos: *id* (clave primaria), *nombre*, *precio*, *cantidad*.
- Insertar **3 registros de prueba** en la tabla *PRODUCTOS*.
- Implementar **un método que devuelva una lista** con todos los productos de la tabla.
- Implementar **un método para insertar un nuevo producto** en la tabla.
- Implementar **un método para actualizar el stock** de un producto según su id.
- Implementar la **conexión a la base de datos** como un *singleton*.
- Utilizar **sentencias preparadas** (*PreparedStatement*)

Las **clases** que debes crear son:

- Clase *Producto* (entidad)
- Clase *ConexionDB* (clase que encapsula la conexión a la DB)

- interfaz `ProductoDAO` (creación de métodos, pero sin implementar)
- Clase `ProductoDAOImpl` (implementará los métodos de la interfaz `ProductoDAO`)
- Clase `ProductoService` (llamará a `ProductoDAOImpl` y hará uso de sus métodos desde el lado de la lógica de negocio).

2. ¿Dónde se lleva a cabo?

Entorno de Eclipse IDE

3. Tiempo de dedicación

2 horas

4. Condición

Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío 🙌

- Clase `Producto`

```
public class Producto {  
  
    private int id;  
    private String nombre;  
    private double precio;  
    private int cantidad;  
  
    // Métodos getter y setter  
  
}
```

- Clase `ConexionDB`

```
public class ConexionDB {  
  
    private static ConexionDB instancia;  
  
    private Connection conn;  
  
    private ConexionDB() {  
        // obtener conexión  
    }  
  
    public static ConexionDB getInstance() {  
        if(instancia == null) {  
            instancia = new ConexionDB();  
        }  
        return instancia;  
    }  
  
    public Connection getConnection() {  
        return this.conn;  
    }  
}
```

```
public void closeConnection() {  
    try {  
        this.conn.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

- Interfaz `ProductoDAO`

```
public interface ProductoDAO {  
  
    public void crearTablaProductos();  
  
    public void insertarProducto(Producto p);  
  
    public List<Producto> obtenerProductos();  
  
    public void actualizarStock(int id, int cantidad);  
  
}
```

- Clase `ProductoDAOImpl`

```
public class ProductoDAOImpl implements ProductoDAO {  
  
    private Connection conn; // conexión singleton JDBC  
  
    public ProductoDAOImpl() {  
        // obtener conexión  
    }  
  
    @Override  
    public void crearTablaProductos() {  
  
        String sql = "CREATE TABLE Productos(" +  
            "id INT PRIMARY KEY," +  
            "nombre VARCHAR(50)," +  
            "precio DECIMAL(10,2)," +  
            "cantidad INT)";  
  
        try {  
            Statement stmt = conn.createStatement();  
            stmt.executeUpdate(sql);  
        } catch (SQLException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```

    }

    @Override
    public void insertarProducto(Producto p) {

        String sql = "INSERT INTO Productos VALUES(?,?,?,?)";

        try {
            PreparedStatement ps = conn.prepareStatement(sql);
            ps.setInt(1, p.getId());
            ps.setString(2, p.getNombre());
            ps.setDouble(3, p.getPrecio());
            ps.setInt(4, p.getCantidad());
            ps.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

```

        return productos;
    }

    @Override
    public void actualizarStock(int id, int cantidad) {

        String sql = "UPDATE Productos SET cantidad = ? WHERE id = ?";

        try {
            PreparedStatement ps = conn.prepareStatement(sql);
            ps.setInt(1, cantidad);
            ps.setInt(2, id);
            ps.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }

    }
}

```

Debes tener en cuenta que para esta clase `ProductoDAOImpl`, es necesario que **conectes a la base de datos**, por lo cuál el código podría verse así:

```
public class ProductoDAOImpl implements ProductoDAO {

    ConexionDB connDB = ConexionDB.getInstance();

    Connection conn = connDB.getConnection();

    // código CRUD

    connDB.closeConnection();
}
```

Donde el comentario `//código CRUD` hace referencia a las imágenes de código anteriores, realizando las operaciones necesarias.

- Clase `ProductoService`

```
public class ProductoService {

    private ProductoDAO productoDAO;

    public ProductoService(ProductoDAO productoDAO) {
        this.productoDAO = productoDAO;
    }

    public void crearTablaProductos() {
        this.productoDAO.crearTablaProductos();
    }

    public void insertarProducto(Producto p) {
        this.productoDAO.insertarProducto(p);
    }

    public List<Producto> obtenerProductos() {
        return this.productoDAO.obtenerProductos();
    }

    public void actualizarStock(int id, int cantidad) {
        this.productoDAO.actualizarStock(id, cantidad);
    }
}
```

De esta forma `ProductoService` está **desacoplado** de la clase concreta `ProductoDAOImpl` y se basa únicamente en la interfaz `ProductoDAO`.

Finalmente, podemos ver cómo quedaría un ejemplo de llamado a los métodos `insertarProducto(p)` y `obtenerProductos()` en la clase que maneja los objetos (en este caso puede ser la Main class).

- Creación de objetos y uso

```
ProductoDAO productoDAO = new ProductoDAOImpl();

ProductoService productoService = new ProductoService(productoDAO);

productoService.crearTablaProductos();

Producto p = new Producto();
p.setNombre("Teclado");
productoService.insertarProducto(p);

List<Producto> productos = productoService.obtenerProductos();

// etc...
```